



UNIVERSITÀ DEGLI STUDI DI SALERNO

**Dipartimento di Scienze Aziendali**  
**Management & Innovation Systems**

Corso di Laurea Magistrale in Data Science e Gestione dell'Innovazione

**Esame di**  
*Massive Data Mining*

**TMDB 5000 Movie Dataset Title Recommendation System**

Bruno Maria Di Maio

matr. 0222800149

**23 aprile 2026**

# Sommario

|                                                  |    |
|--------------------------------------------------|----|
| Introduzione.....                                | 3  |
| STRUMENTI IMPIEGATI .....                        | 3  |
| Analisi Esplorativa dei Dati (EDA).....          | 5  |
| CARICAMENTO ED ESPLORAZIONE DEL DATASET .....    | 5  |
| PULIZIA DEL DATASET .....                        | 7  |
| PROCESSING DATASET .....                         | 8  |
| VISUALIZZAZIONE .....                            | 9  |
| Distribuzioni .....                              | 9  |
| Boxplot.....                                     | 11 |
| Grafici a barre.....                             | 12 |
| Cartogramma .....                                | 14 |
| RELAZIONI NUMERICHE .....                        | 15 |
| Matrice di Correlazione .....                    | 15 |
| Scatter Plot Budget vs Revenue.....              | 15 |
| Grafico a linee.....                             | 16 |
| Realizzazione del progetto.....                  | 17 |
| APPROCCIO TEORICO .....                          | 17 |
| Distanza di Levenshtein.....                     | 17 |
| Cosine Similarity .....                          | 17 |
| Definizione dei parametri di configurazione..... | 17 |
| La scelta del modello.....                       | 19 |
| CARICAMENTO DEL DATASET ED HELPER FUNCTIONS..... | 19 |
| extract_names.....                               | 19 |
| extract_list.....                                | 20 |
| get_director .....                               | 20 |
| fuzzy_title_score .....                          | 20 |
| fuzzy_title_score_strict.....                    | 21 |
| normalize .....                                  | 21 |
| PREPROCESSING DEL DATASET .....                  | 22 |
| ENCODING DEL DATABASE .....                      | 23 |
| FUNZIONE DI RICERCA .....                        | 24 |
| ESECUZIONE.....                                  | 26 |
| VALIDAZIONE.....                                 | 28 |
| Metriche di valutazione .....                    | 29 |
| Conclusioni.....                                 | 31 |

## Introduzione

Il presente elaborato illustra il progetto realizzato per l'esame di Massive Data Mining, incentrato sul dataset TMDb 5000 Movie Dataset, una raccolta di informazioni su 5000 film tra cui titoli, generi, budget, incassi, cast, regia e sinossi.

L'obiettivo principale del progetto è la realizzazione di un sistema di raccomandazione cinematografica in grado di restituire, a partire da una query dell'utente, che può essere il titolo di un film anche scritto in modo approssimato, oppure una semplice descrizione a testo libero, i 10 candidati più pertinenti presenti nel dataset. La sfida principale consiste nel gestire in modo robusto l'imprecisione dell'input: errori di battitura, titoli parziali e query descrittive rappresentano scenari reali e frequenti che un sistema di ricerca efficace deve saper affrontare.

A tal fine sono stati esplorati e messi a confronto due approcci distinti: il fuzzy matching, basato sulla distanza di Levenshtein tramite la libreria TheFuzz, e la ricerca basata su cosine similarity, realizzati entrambi tramite embedding vettoriali generati dal modello all-mpnet-base-v2 della libreria Sentence Transformers. Prima di procedere alla realizzazione del sistema di ricerca, è stata condotta un'Analisi Esplorativa dei Dati (EDA) sul dataset, con l'obiettivo di comprenderne la struttura, individuare e gestire i valori mancanti, e ricavare insight significativi su distribuzione dei budget, degli incassi, delle durate, dei generi e delle tendenze produttive nel tempo.

## Strumenti impiegati

Per perseguire l'intento sopracitato è stato utilizzato il linguaggio di programmazione Python tramite un Notebook Jupyter. Sono state impiegate le seguenti librerie:

- ast per leggere stringhe che sembrano liste o dizionari e trasformarle in oggetti
- numpy per calcoli numerici
- pandas per lavorare con tabelle di dati
- la classe SentenceTransformer dalla libreria sentence\_transformers per usare un modello di intelligenza artificiale in grado di trasformare una frase in un vettore numerico che ne rappresenta il significato
- la funzione cosine\_similarity dalla libreria scikit-learn per misurare quanto due vettori numerici puntano nella stessa direzione, restituendo un valore tra -1 e 1
- il modulo fuzz dalla libreria thefuzz per avere funzioni di confronto tra stringhe di testo anche quando non sono identiche (fuzzy matching)

Inoltre, per la realizzazione di una prima Analisi Esplorativa dei Dati (EDA) sono state utilizzate, in combinazione con alcune delle sopracitate, le librerie:

- matplotlib per la creazione di grafici statici
- seaborn per la creazione di grafici statistici con un'interfaccia migliore
- il modulo plotly.express della libreria plotly che permette di creare grafici interattivi
- la funzione Counter dalla libreria collections che consente di contare occorrenze di elementi in un iterabile restituendo un dizionario

## Analisi Esplorativa dei Dati (EDA)

### Caricamento ed Esplorazione del dataset

Prima di passare nel vivo del lavoro svolto è utile un'analisi dei dati che ci permetta di conoscere al meglio il dataset che stiamo per manipolare. Dopo aver caricato i file 'tmdb\_5000\_movies.csv' e 'tmdb\_5000\_credits.csv' ed averne eseguito il merge tramite l'id univoco del film possiamo osservare la "forma" del dataset. Scopriamo che questo contiene 4803 righe e 24 colonne. Queste ultime sono:

| Nome Colonna                | Descrizione                                           | Tipo di Dato |
|-----------------------------|-------------------------------------------------------|--------------|
| <b>budget</b>               | investimento in dollari per la realizzazione del film | int64        |
| <b>genres</b>               | elenco dei vari generi di un film                     | object       |
| <b>homepage</b>             | sito web del film                                     | object       |
| <b>id</b>                   | identificativo univoco del film                       | int64        |
| <b>keywords</b>             | parole chiave che contraddistinguono un film          | object       |
| <b>original_language</b>    | lingua originale del film                             | object       |
| <b>original_title</b>       | titolo in lingua originale del film                   | object       |
| <b>overview</b>             | sinossi che descrive il film                          | object       |
| <b>popularity</b>           | indice di polarità del prodotto                       | float64      |
| <b>production_companies</b> | lista delle aziende produttrici del film              | object       |
| <b>production_countries</b> | Paesi contribuenti alla produzione del film           | object       |
| <b>release_date</b>         | data di rilascio del film                             | object       |
| <b>revenue</b>              | incassi in dollari del film                           | int64        |
| <b>runtime</b>              | durata in minuti del film                             | float64      |
| <b>spoken_languages</b>     | lingue parlate all'interno del film                   | object       |
| <b>status</b>               | stato di rilascio del film                            | object       |
| <b>tagline</b>              | sottotitolo del film                                  | object       |
| <b>title_x</b>              | titolo del film                                       | object       |

|                     |                                                 |         |
|---------------------|-------------------------------------------------|---------|
| <b>vote_average</b> | media dei voti del film                         | float64 |
| <b>vote_count</b>   | quanti voti sono stati dati al film             | int64   |
| <b>movie_id</b>     | duplicato di 'id'                               | int64   |
| <b>title_y</b>      | duplicato di 'title_x'                          | object  |
| <b>cast</b>         | elenco del cast di un film                      | object  |
| <b>crew</b>         | elenco delle persone che hanno lavorato al film | object  |

Per un'analisi dei dati numerici del film possiamo usare la funzione `describe` di pandas che ci permette di vedere la seguente tabella.

|       | budget       | id            | popularity  | revenue      | runtime     | vote_average | vote_count   | movie_id      |
|-------|--------------|---------------|-------------|--------------|-------------|--------------|--------------|---------------|
| count | 4.803000e+03 | 4803.000000   | 4803.000000 | 4.803000e+03 | 4801.000000 | 4803.000000  | 4803.000000  | 4803.000000   |
| mean  | 2.904504e+07 | 57165.484281  | 21.492301   | 8.226064e+07 | 106.875859  | 6.092172     | 690.217989   | 57165.484281  |
| std   | 4.072239e+07 | 88694.614033  | 31.816650   | 1.628571e+08 | 22.611935   | 1.194612     | 1234.585891  | 88694.614033  |
| min   | 0.000000e+00 | 5.000000      | 0.000000    | 0.000000e+00 | 0.000000    | 0.000000     | 0.000000     | 5.000000      |
| 25%   | 7.900000e+05 | 9014.500000   | 4.668070    | 0.000000e+00 | 94.000000   | 5.600000     | 54.000000    | 9014.500000   |
| 50%   | 1.500000e+07 | 14629.000000  | 12.921594   | 1.917000e+07 | 103.000000  | 6.200000     | 235.000000   | 14629.000000  |
| 75%   | 4.000000e+07 | 58610.500000  | 28.313505   | 9.291719e+07 | 118.000000  | 6.800000     | 737.000000   | 58610.500000  |
| max   | 3.800000e+08 | 459488.000000 | 875.581305  | 2.787965e+09 | 338.000000  | 10.000000    | 13752.000000 | 459488.000000 |

Qui abbiamo un riepilogo statistico con diversi parametri interessanti:

- count: Numero di valori non nulli
- mean: Media aritmetica
- std: Deviazione standard
- min: Valore minimo
- 25%: Primo quartile (Q1)
- 50%: Mediana (Q2)
- 75%: Terzo quartile (Q3)
- max: Valore massimo

Dopo aver letto diverse statistiche del nostro dataframe possiamo anche visualizzarle, andando a vedere, ad esempio, i valori mancanti per quelle colonne che ne hanno. Questo lo facciamo andando a sommare i vari valori 'null' delle colonne e filtrando per vedere solo quelli che ci interessano. Possiamo poi metterli su un istogramma in cui verifichiamo la percentuale per ogni colonna dei valori mancanti.

```
homepage      3091
tagline       844
overview       3
runtime        2
release_date   1
dtype: int64
```

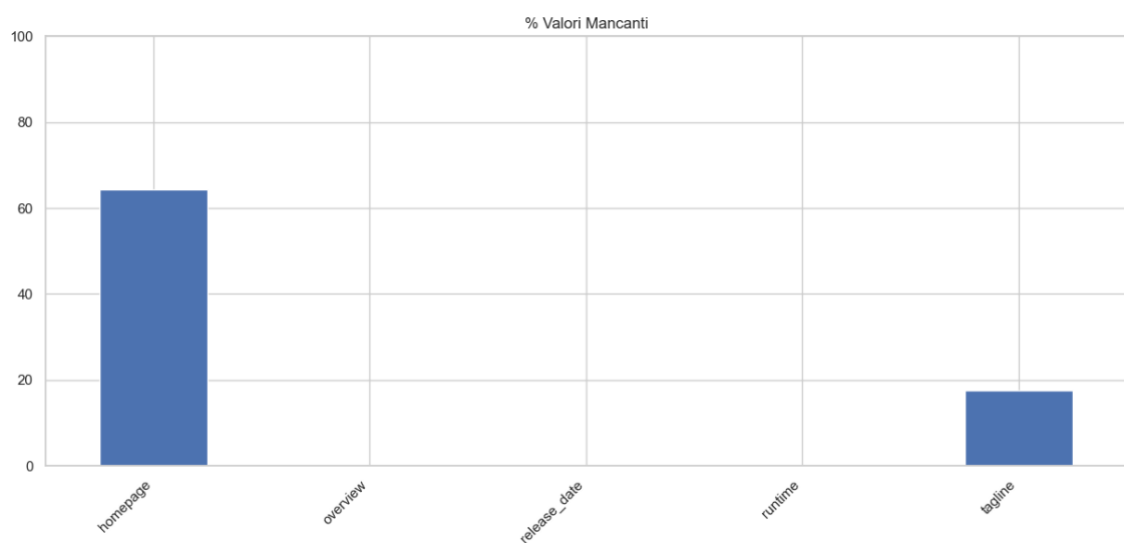
```

missing = df.isnull().sum().sort_values(ascending=False)
print(missing[missing > 0])

missing_pct = (df.isnull().sum() / len(df)) * 100
missing_pct[missing_pct > 0].plot(kind="bar", figsize=(15, 6), title="% Valori Mancanti")
plt.ylim(0, 100)
plt.xticks(rotation=45, ha="right")
plt.show()

```

Qui possiamo osservare che le colonne che presentano più valori mancanti sono ‘homepage’ (>60%) e ‘tagline’ (>15%).



Infine, possiamo verificare che all'interno del dataset non sono presenti righe duplicate tramite il comando ‘df.duplicated().sum()’.

### **Pulizia del dataset**

Dopo aver visualizzato e capito come funziona il dataset possiamo finalmente passare ad una fase di pulizia in cui formattiamo le varie colonne nel formato giusto per l'analisi ed eliminiamo i valori non utili.

In primis definiamo le funzioni che ci servono alla manipolazione dei dati:

- la funzione ‘extract\_list’ estrae i nomi da una colonna JSON-like e li riporta sotto forma di lista, con la possibilità di applicare un limite al numero di nomi che vogliamo estrarre.
- la funzione ‘get\_director’ estrae il nome del regista dalla colonna JSON-like, in modo da non avere inutilmente tutti i membri della crew che ha lavorato al film.

```

def extract_list(text, limit = None):
    try:
        items = ast.literal_eval(text)
        names = [item["name"] for item in items]
        return names[:limit] if limit else names
    except:
        return []

def get_director(text):
    try:
        items = ast.literal_eval(text)
        for item in items:
            if item["job"] == "Director":
                return item["name"]
        return ""
    except:
        return ""

```

## Processing Dataset

Per processare il dataset vengono eseguite diverse operazioni in modo da pulirlo e renderlo più comodo per eseguire la vera e propria ricerca. In primo luogo, viene estratto l'anno di pubblicazione del film dal campo `release_date` in cui abbiamo una data in formato YYYY-MM-DD ed inserito nella colonna `year`. In questo modo possiamo condurre analisi temporali basate sugli anni di produzione dei film. Elimino le colonne per cui non è stato rilasciato il film controllando la colonna `status` per la stringa "Released", poiché potrebbero contenere dati mancanti o inesatti. Vengono poi eliminate tutte quelle colonne che non sono utili alla nostra analisi, ovvero: `title_y` (duplicata in seguito al merge), `tagline`, `homepage`, `movie_id` (usiamo la colonna `id` al suo posto), `overview` e `status`. Eliminiamo inoltre i casi in cui `runtime` non ha valore visto che, essendo solo 2, sono trascurabili. Rinominiamo poi la colonna `title_x` per farla tornare al nome originale `title`. Filtriamo inoltre i valori di `budget`, `revenue` e `vote_average` per avere solo le colonne maggiori di 0 e creiamo la nuova colonna `roi` che corrisponde, appunto, al Return Of Investment, calcolato con la formula  $ROI = \frac{revenue - budget}{budget} \cdot 100$ . Successivamente viene ricavato il nome del regista semplicemente applicando alla colonna `crew` la funzione `get_director` definita precedentemente e riportando il risultato all'interno della nuova colonna `director`. Allo stesso modo estraiamo i nomi di cast, genres, keywords, `production_countries` e `production_companies` con la funzione `extract_list`. Viene poi sostituito il contenuto delle righe vuote nelle colonne che contengono stringhe con stringhe vuote per evitare errori durante la ricerca o l'utilizzo di qualche funzione. Infine, reimpostiamo gli indici del dataframe per evitare il disallineamento degli indici. Dopo la pulizia possiamo notare che il dataframe ha ora **20 colonne e 3226 righe**.



```

df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")
df["year"] = df["release_date"].dt.year.astype("Int64")

df = df[df["status"] == "Released"]

df = df.drop(["title_y", "tagline", "homepage", "movie_id", "overview", "status"], axis = 1)
df = df.dropna(subset=["runtime"])
df = df.rename(columns={"title_x": "title"})

df = df[(df["budget"] > 0) & (df["revenue"] > 0) & (df["vote_average"] > 0)]
df["roi"] = (df["revenue"] - df["budget"]) / df["budget"] * 100

df["director"] = df["crew"].apply(get_director)
df["cast"] = df["cast"].apply(lambda x: extract_list(x, limit=5))
for col in ["genres", "keywords", "production_countries", "production_companies"]:
    df[col] = df[col].apply(extract_list)

for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].fillna("")

df = df.reset_index(drop=True)

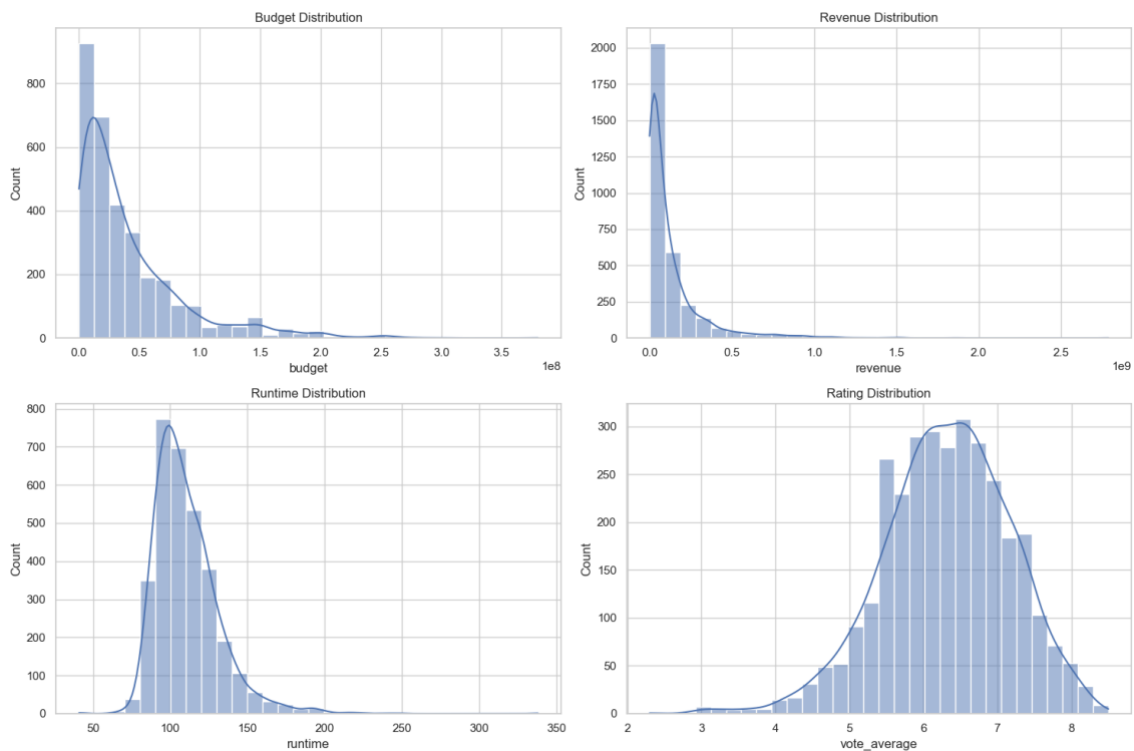
df.shape

```

## Visualizzazione

### Distribuzioni

Possiamo finalmente passare ad una fase di visualizzazione effettiva dei dati, in primis con una distribuzione tramite istogrammi con curva KDE (Kernel Density Estimation) dei parametri di Budget, Revenue, Runtime e Rating.

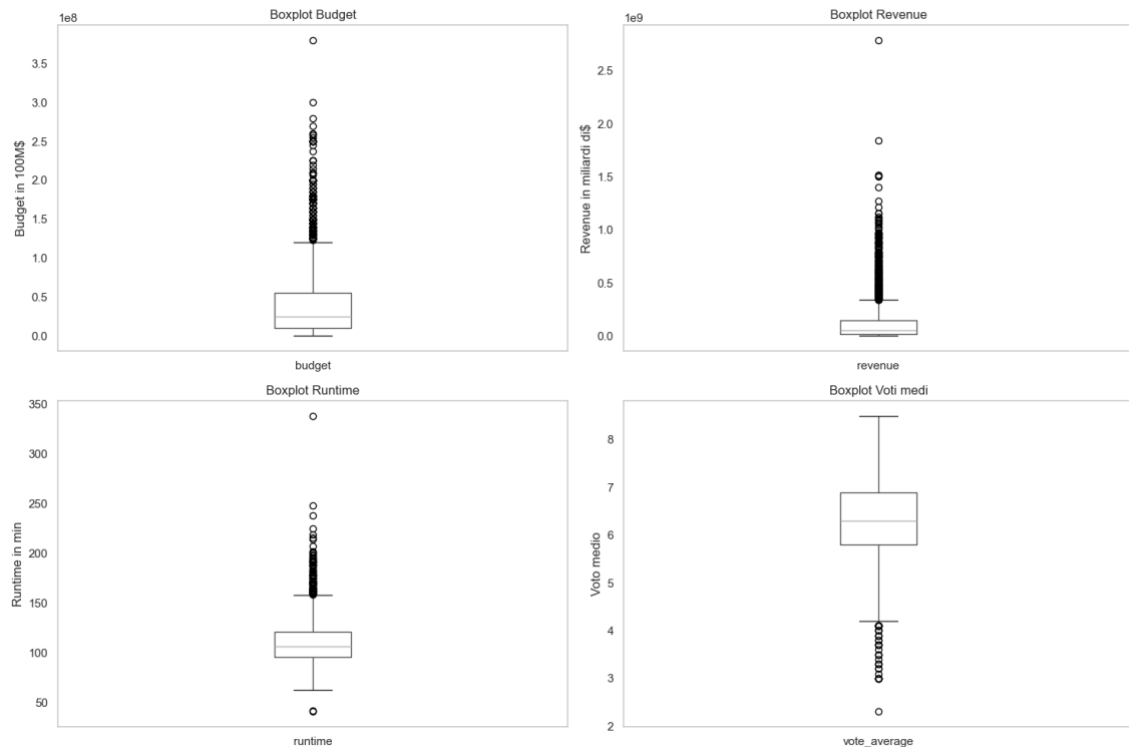


Da questi grafici possiamo notare che:

- la distribuzione del budget è fortemente asimmetrica a destra. La stragrande maggioranza dei film ha un budget concentrato tra 0 e ~50 milioni di dollari, con un picco attorno ai 10–20M\$. Pochissimi film raggiungono budget superiori a 200M\$, evidenziando che le produzioni ad alto budget sono eccezioni rare.
- anche la distribuzione dei ricavi è molto asimmetrica a destra, ancora più pronunciata rispetto al budget. La quasi totalità dei film incassa meno di 500 milioni di dollari, con una coda lunghissima che si estende fino a ~2.7 miliardi, quasi certamente riferibile a grandi blockbuster.
- la distribuzione della durata dei film è approssimativamente normale con lieve asimmetria a destra. Il picco è intorno ai 90–100 minuti, che rappresenta la durata standard per la maggior parte delle produzioni. La coda destra mostra film con durate eccezionali oltre i 200–300 minuti.
- la distribuzione dei voti medi è la più simmetrica delle quattro. Il picco si trova attorno a 6–6,5, con distribuzione che si estende da 2 a ~8,5. La coda sinistra verso i voti bassi (0–4) è relativamente sottile, suggerendo che la maggior parte dei film riceve valutazioni nella fascia media.

## Boxplot

Passiamo poi ad una visualizzazione tramite boxplot dei parametri Budget, Revenue, Runtime e Rating.



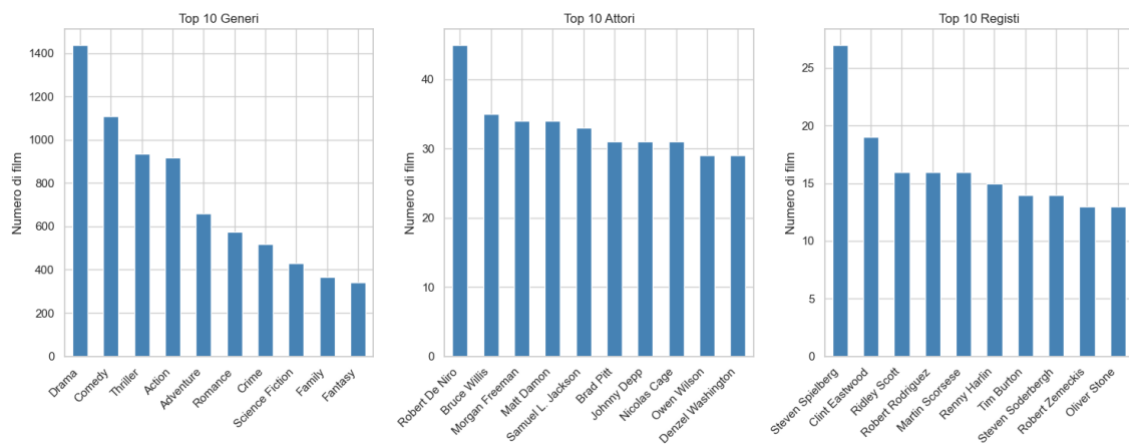
Qui è possibile trarre diverse conclusioni:

- il boxplot è compreso tra ~5M\$ e ~50M\$, con la mediana attorno ai 20–25M\$. Il whisker superiore arriva a circa 120M\$, ma la presenza di numerosi outlier fino a quasi 380M\$ conferma la forte asimmetria già osservata nell'istogramma. Il 50% centrale dei film ha dunque un budget relativamente contenuto
- la distribuzione dei ricavi è ancora più compressa il boxplot è strettissimo tra ~0 e ~120M\$, con la mediana vicinissima allo zero, a indicare che la metà dei film incassa pochissimo. Gli outlier sono però molto estremi, con un massimo isolato attorno a 2.7 miliardi di dollari. Il whisker superiore si ferma a ~350M\$, e la nuvola di punti al di sopra rappresenta blockbuster eccezionali.
- la durata dei film è la variabile più compatta e simmetrica tra le quattro. Il boxplot va da circa 95 a 120 minuti, con la mediana intorno ai 110 minuti. Gli outlier superiori arrivano fino a ~330 minuti, mentre il valore minimo del whisker inferiore è circa 63 minuti. La scatola ristretta conferma che la maggior parte dei film rispetta la durata standard hollywoodiana.

- il boxplot dei voti medi è il più regolare e centrato. Il boxplot va da circa 5.8 a 6.8, con la mediana attorno a 6.3–6.4. I whisker si estendono da ~4.0 a ~8.0, mentre gli outlier bassi scendono fino a poco più di 2.

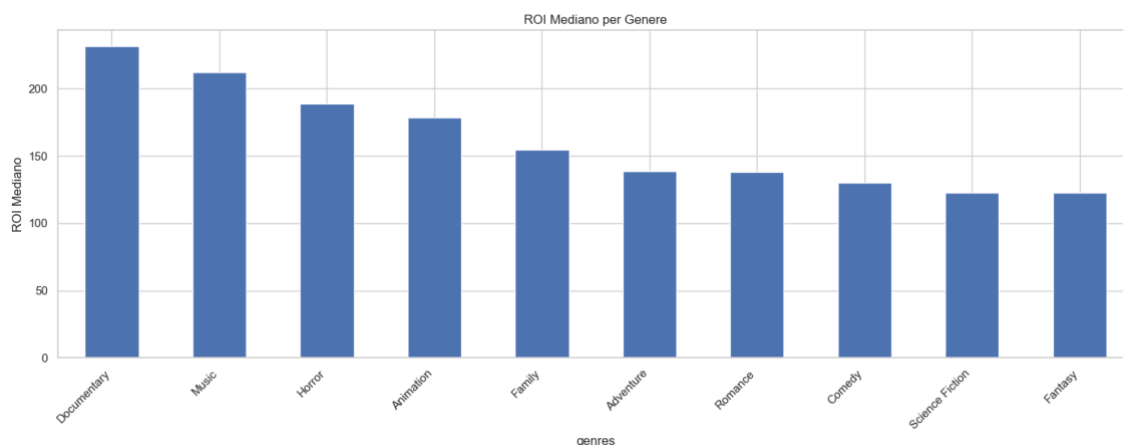
### Grafici a barre

Grazie ai grafici a barre possiamo vedere le frequenze categoriali del dataset, ovvero quante volte ciascuna categoria appare nel dataset. A differenza degli istogrammi classici, questi grafici operano su variabili categoriali, ordinate in modo decrescente per evidenziare immediatamente i valori dominanti.

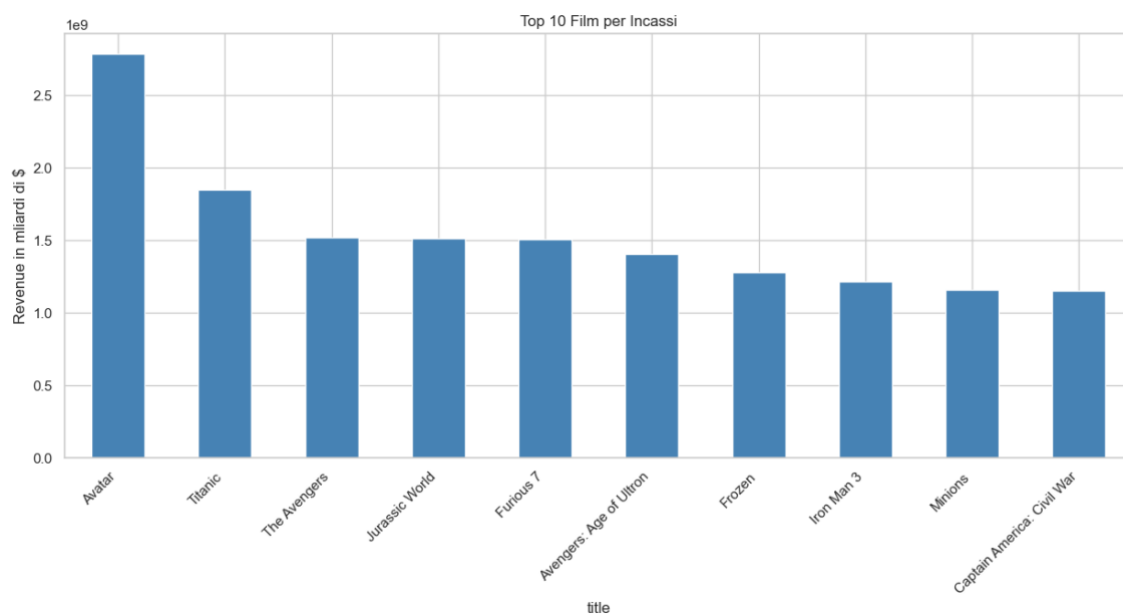


Da questi grafici possiamo vedere che:

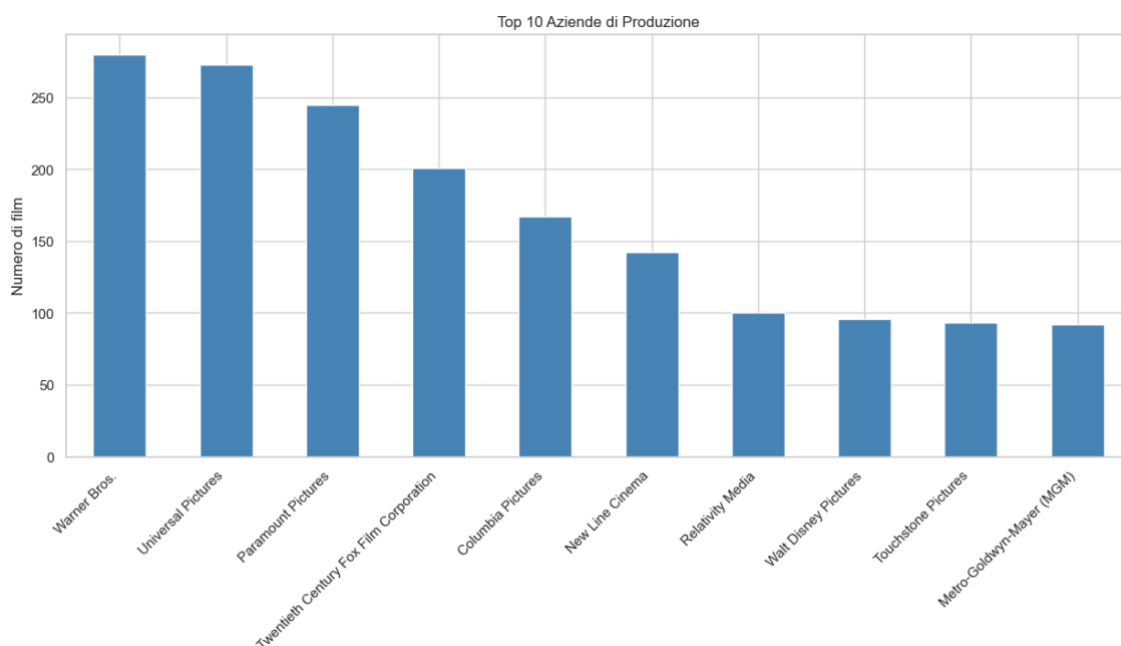
- Il Drama è di gran lunga il genere più rappresentato, con 1441 film, seguito da Comedy (1110) e Thriller (935). Action occupa il quarto posto (918), mentre i generi meno rappresentati tra i top 10 sono Family (365) e Fantasy (342). Il distacco tra Drama e il resto suggerisce una netta dominanza di questo genere nel dataset.
- Robert De Niro è in testa alla classifica degli attori con 45 film, staccando nettamente il secondo classificato Bruce Willis (35). Da lì in poi l'appiattimento della top 10 è abbastanza evidente con film che vanno da 35 a 29 film, formando un gruppo molto compatto.
- Steven Spielberg è il regista più menzionato con 27 film, seguito da Clint Eastwood (19). Martin Scorsese, Robert Rodriguez e Ridley Scott sono raggruppati a 16 film. La classifica mostra registi con carriere molto prolifiche, tutti con un minimo di 13 film nel dataset.



Il ritorno dell'investimento è un fattore importantissimo, e possiamo analizzare quali generi producono un maggior ROI tramite questo grafico a barre. Vediamo dunque che il genere Documentary è quello che frutta maggiormente, seguito da Music e Horror, nonostante non siano questi tra i più prodotti nel nostro dataset.



Qui possiamo analizzare i film che hanno incassato maggiormente all'interno del nostro dataset, con Avatar al primo posto, seguito da Titanic e The Avengers, fino ad arrivare al decimo posto con Captain America: Civil War.

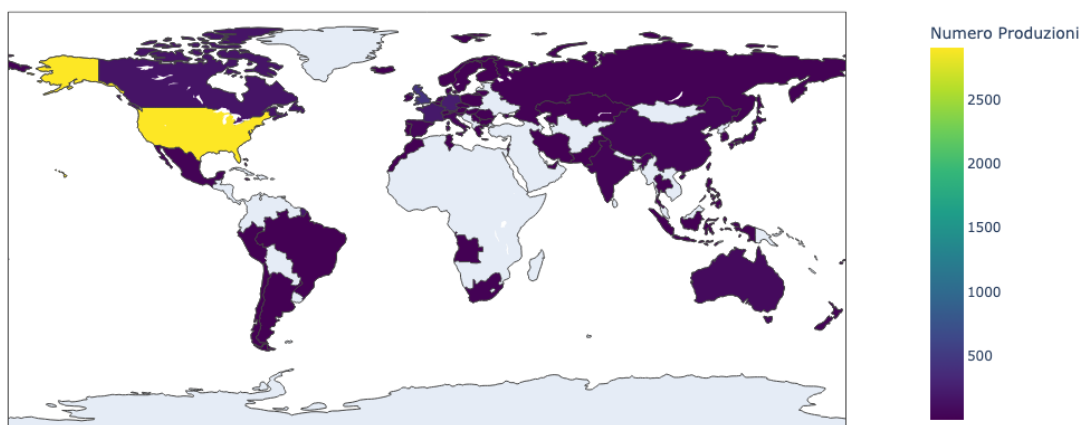


In questo grafico a barre possiamo notare, invece, le 10 più grandi Aziende di Produzione presenti all'interno del dataset. Nelle prime tre posizioni troviamo Warner Bros. con 280 film, Universal Pictures con 273 film e Paramount Pictures con 245 film. Da lì in poi la classifica va sempre più a calare ed appiattirsi arrivando alle ultime 4 posizioni tra i 100 e 92 film.

### Cartogramma

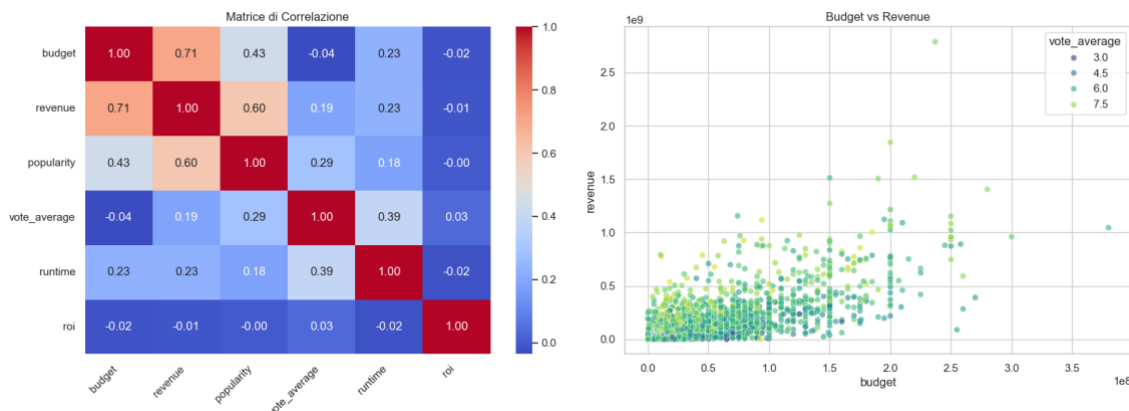
Tramite questo grafico possiamo individuare i principali Paesi di produzione dei film presenti nel dataset. Qui possiamo notare un netto distacco degli Stati Uniti d'America con 2906 film, che si lascia molto dietro il secondo posto preso dal Regno Unito con 436 film. Da lì in poi i Paesi vanno sempre a calare in quanto a numero di produzione, compresa l'Italia che comunque si trova al sesto posto su 61 Paesi con 48 produzioni.

Produzioni per Paese



## Relazioni numeriche

Questi due grafici analizzano le relazioni tra le variabili numeriche del dataset: la matrice di correlazione fornisce una visione globale delle dipendenze lineari, mentre lo scatter plot approfondisce il rapporto specifico tra budget e revenue.



## Matrice di Correlazione

La matrice riporta i coefficienti di correlazione di Pearson tra le variabili budget, revenue, popularity, vote\_average, runtime e roi. Le correlazioni più rilevanti sono:

- budget e revenue: 0.71, correlazione positiva forte, quindi i film con budget più alto tendono a incassare di più
- revenue e popularity: 0.60, i film più popolari generano ricavi maggiori
- budget e popularity: 0.43, correlazione moderata, film costosi sono generalmente più noti
- runtime e vote\_average: 0.39, i film più lunghi tendono ad avere voti leggermente più alti
- vote\_average e budget: -0.04, correlazione praticamente nulla, quindi spendere di più non garantisce un voto migliore
- roi: praticamente incorrelato con tutte le variabili, suggerendo che il ritorno sull'investimento dipende da fattori non lineari o non presenti nel dataset

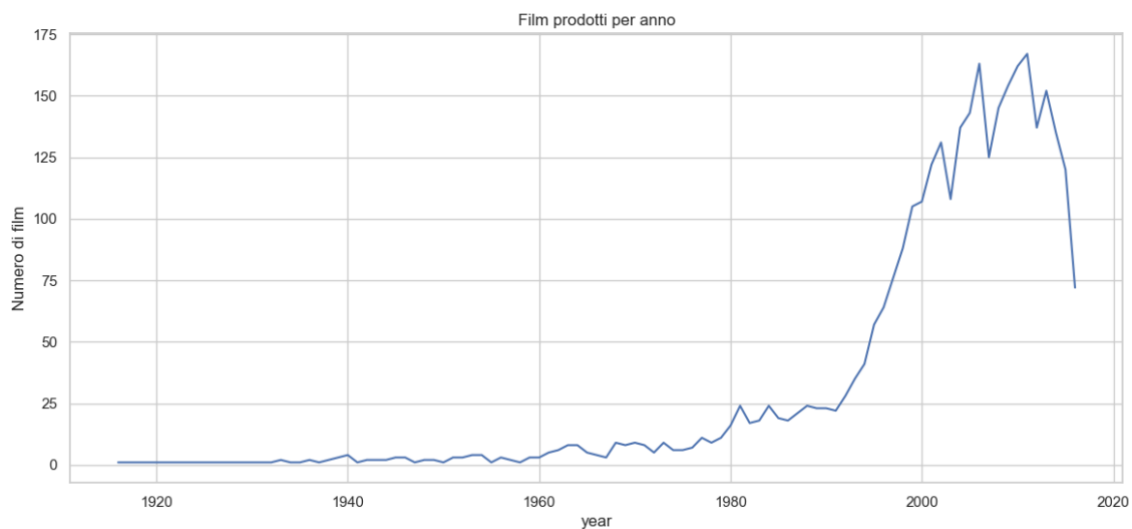
## Scatter Plot Budget vs Revenue

Il grafico mostra la relazione diretta tra budget e revenue, con i punti colorati in base al vote\_average. Si osserva una certa tendenza positiva all'aumentare del budget cresce mediamente anche la revenue, ma con una dispersione molto elevata, specialmente nella fascia 0–100M\$ di budget dove i risultati sono estremamente variabili. I film con vote\_average più

alto (punti giallo-verdi) tendono a concentrarsi nelle fasce di budget medio-alte, e alcuni di questi sono outlier con revenue superiore a 2.5 miliardi.

### Grafico a linee

Grazie a questo grafico a linee possiamo analizzare le tendenze di produzione annuali dei film presenti nel dataset, che vedono gli anni di produzione schizzare tra gli anni '90 ed il 2000 con il picco di produzione raggiunto intorno al 2010/2015. Tuttavia, dopo il 2000, la linea diventa molto più irregolare, indicando forti variazioni annuali nella produzione rispetto alla stabilità dei decenni precedenti. Intorno al 2017 abbiamo un calo drastico della produzione, questo calo drastico non indica necessariamente un collasso dell'industria cinematografica, ma è quasi certamente dovuto a un limite del dataset.





## **Realizzazione del progetto**

### **Approccio teorico**

Per realizzare il progetto, oltre agli strumenti utilizzati, sono stati applicati due approcci teorici basati rispettivamente sulla Distanza di Levenshtein e sulla Cosine Similarity. In questo caso si tratta di approcci che sfruttano la fuzzy search. La fuzzy search è una tecnica di ricerca in grado di trovare corrispondenze anche quando la query di ricerca non corrisponde perfettamente ai relativi dati. Non si limita a una corrispondenza letterale tra i caratteri, ma identifica i risultati simili alla query di ricerca in termini di ortografia, significato o altri criteri. Questo può essere particolarmente utile quando si ha a che fare con input utente, che possono includere errori di battitura, variazioni (plurale o singolare, abbreviazioni, stemming e altro) e altre incoerenze dovute ai diversi modi in cui gli utenti comunicano.

La fuzzy search utilizza vari algoritmi e tecniche per determinare la somiglianza tra due stringhe di testo, la query di ricerca e la potenziale corrispondenza nei dati.

#### **Distanza di Levenshtein**

All'interno dell'approccio basato sulla libreria `thefuzz` viene utilizzata la distanza di Levenshtein, che è una misura di differenza tra due stringhe, la quale determina il numero minimo di modifiche (come inserimenti, eliminazioni o sostituzioni) necessarie per trasformare una stringa in un'altra. Una distanza di Levenshtein più bassa indica una maggiore somiglianza. Ad esempio, le parole "bar" e "biro" hanno una distanza di Levenshtein pari a 2.

#### **Cosine Similarity**

L'approccio basato invece sulla Cosine Similarity si basa su una metrica di somiglianza che determina la somiglianza di due punti dati in base alla direzione in cui puntano anziché alla loro lunghezza o dimensione. Il calcolo della similarità del coseno richiede la misurazione del coseno dell'angolo tra due vettori diversi da zero in uno spazio di prodotto interno. Questa misurazione produce un punteggio di somiglianza del coseno, con valori che variano da -1 a 1.

#### **Definizione dei parametri di configurazione**

Per rendere il progetto il più possibile flessibile sono stati definiti alcuni parametri di configurazione modificabili in modo dinamico in modo da gestire al meglio il funzionamento del programma. In particolare abbiamo:

- TOP: definisce il numero di film da mostrare alla fine
- CAST\_LIMIT: indica quanti membri del cast preservare dalla pulizia

- MODEL\_NAME: ci dice quale modello di Sentence Transformer prendere
- TITLE\_THRESHOLD: definisce il livello di accuratezza del titolo da considerare per la modalità ricerca tramite titolo
- WEIGHT\_TITLE\_HIGH: indica il peso dello score del titolo sulla query in caso di ricerca per titolo
- WEIGHT\_TITLE\_LOW: indica il peso dello score del titolo sulla query in caso di ricerca per descrizione

```
TOP = 10
CAST_LIMIT = 5
MODEL_NAME = "all-mpnet-base-v2"

TITLE_THRESHOLD = 0.65
WEIGHT_TITLE_HIGH = 0.70
WEIGHT_TITLE_LOW = 0.25
```

Inoltre, vengono elencate le queries che verranno usate per testare gli algoritmi, con i corrispondenti film associati che dovrebbero trovare.

```
queries = [
    # Titoli Completi
    ("Blade Runner", "Blade Runner"),
    ("The Dark Knight Rises", "The Dark Knight Rises"),
    ("Raging Bull", "Raging Bull"),
    ("Robocop", "Robocop"),
    # Titoli parziali o scritti male
    ("goffather", "The Godfather"),
    ("ghost", "Ghost"),
    ("lord of the rings", "The Lord of the Rings: The Fellowship of the Ring"),
    ("montecristo", "The count of monte cristo"),
    ("the mtrax", "The Matrix"),
    ("big short", "The Big Short"),
    # Descrizioni
    ("science fiction movie with light sabers", "Star Wars"),
    ("animated movie about a rat who cooks", "Ratatouille"),
    ("superhero with iron suit", "Iron Man"),
    ("killer shark on a beach town", "Jaws"),
    ("space crew finds alien eggs on a planet", "Alien"),
    ("mel brooks parody of star wars with spaceships", "Spaceballs"),
    ("time travel DeLorean 1985", "Back to the Future"),
    ("tributes are chosen to compete in killing games", "The Hunger Games"),
    ("horror movie with a masked killer", "Halloween"),
    ("cyborg travels into the past to kill", "The Terminator"),
    # Attori
    ("emma stone, ryan gosling and steve carell", "Crazy, Stupid, Love."),
    ("matt damon and robin williams", "Good Will Hunting"),
    ("ryan gosling as a stuntman", "Drive"),
    ("margot robbie in a martin scorsese movie", "The Wolf of Wall Street"),
    ("science fiction movie by john carpenter", "The Thing"),
]
```

## La scelta del modello

Il modello scelto è all-mpnet-base-v2, un modello di sentence embeddings della libreria Sentence Transformers, progettato per convertire frasi e brevi paragrafi in vettori numerici che catturano il significato semantico del testo.

Il modello si basa su MPNet di Microsoft, un sistema che legge e analizza il testo in modo più accurato rispetto ai modelli precedenti come BERT. È stato "allenato" su un miliardo di coppie di frasi imparando a capire quali frasi hanno un significato simile e quali no. Ogni testo in input viene trasformato in un vettore a 768 dimensioni che rappresenta il suo significato. Il limite di input di default è 384 word pieces. Tra i modelli della stessa famiglia, all-mpnet-base-v2 è quello che produce i risultati migliori in termini di qualità. Rispetto a versioni più leggere come all-MiniLM-L6-v2, capisce il testo in modo più preciso, ma è anche più lento e richiede più risorse computazionali.

## Caricamento del dataset ed helper functions

In primis vengono caricati entrambi i file CSV tramite Pandas e vengono uniti tramite l'id univoco di ogni film. Questa operazione viene eseguita per evitare omonimie di film presenti nel database (es. Batman 1989 e Batman 1966).

```
df1 = pd.read_csv("data/tmdb_5000_movies.csv")
df2 = pd.read_csv("data/tmdb_5000_credits.csv")
df = pd.merge(df1, df2, left_on="id", right_on="movie_id")
```

Successivamente vengono definite tutte le funzioni di sostegno alla ricerca e alla pulizia del dataset, ognuna con uno scopo diverso.

### extract\_names

La funzione extract\_names estrae i nomi da una colonna JSON-like e li riporta come semplice testo, con la possibilità di applicare un limite al numero di nomi che vogliamo estrarre.

```
def extract_names(text, limit = None):
    try:
        items = ast.literal_eval(text)
        names = [item["name"] for item in items]
        if limit:
            names = names[:limit]
        return " ".join(names)
    except:
        return ""
```

### **extract\_list**

La funzione `extract_list`, come la precedente, estrae i nomi da una colonna JSON-like, ma riporta sotto forma di lista, anche qui con la possibilità di applicare un limite al numero di nomi che vogliamo estrarre.

```
def extract_list(text, limit = None):
    try:
        items = ast.literal_eval(text)
        names = [item["name"] for item in items]
        return names[:limit] if limit else names
    except:
        return []
```

### **get\_director**

La funzione `get_director` estrae il nome del regista dalla colonna JSON-like, in modo da non avere inutilmente tutti i membri della crew che ha lavorato al film visto che non sono rilevanti ai fini della ricerca.

```
def get_director(text):
    try:
        items = ast.literal_eval(text)
        for item in items:
            if item["job"] == "Director":
                return item["name"]
        return ""
    except:
        return ""
```

### **fuzzy\_title\_score**

La funzione `fuzzy_title_score` sfrutta la libreria `TheFuzz` per lo string matching con la Distanza di Levenshtein. In particolare, la funzione è stata creata per assegnare ad ogni titolo uno score da 0 a 1 in base alla somiglianza con la query che stiamo analizzando. Questa funzione usa i parametri:

- `ratio`: per un confronto carattere per carattere, stringa intera
- `partial_ratio`: con cui cerca la query come sottostringa del titolo (ideale per titoli parziali)
- `token_set_ratio`: grazie al quale si ignorano l'ordine delle parole ed i duplicati
- `token_sort_ratio`: per ordinare le parole alfabeticamente prima di confrontarle

Prende il massimo tra questi valori che normalmente vanno da 0 a 100 e lo divide per 100 per rimanere nell'intervallo `[0, 1]`. Prendere il massimo tra questi valori garantisce che almeno uno dei criteri di matching "colpisca" nel caso migliore. Una limitazione del calcolo della fuzzy

search risiede nel fatto che titoli estremamente corti sono facilmente rintracciabili con qualsiasi ricerca basata sul titolo, per mitigare questo problema viene eseguito un controllo sulla lunghezza della query e del titolo per penalizzare titoli troppo corti su query più lunghe.

```
def fuzzy_title_score(query, title):
    q, t = query.lower(), title.lower()
    score = max(
        fuzz.ratio(q, t),
        fuzz.partial_ratio(q, t),
        fuzz.token_set_ratio(q, t),
        fuzz.token_sort_ratio(q, t),
    ) / 100.0

    if len(t) <= 3 and len(q) > len(t) * 3:
        len_ratio = min(len(t), len(q)) / max(len(t), len(q))
        score *= len_ratio

    return score
```

### **fuzzy\_title\_score\_strict**

La funzione `fuzzy_title_score_strict` è una versione con metriche più "strette" di `fuzzy_title_score`, usata solo per decidere se la query è un titolo o una descrizione, ma non per il ranking finale. In questo caso viene escluso il `partial_ratio` dato che una query descrittiva lunga come "thriller movie with a girl protagonist" contiene parole comuni. Un titolo corto come "The Girl" o "With" viene trovato come sottostringa e ottiene `partial_ratio=100`, facendo scattare erroneamente la modalità titolo. `ratio` e `token_sort_ratio` richiedono invece una somiglianza globale tra le due stringhe, e non si innescano su query lunghe.

```
def fuzzy_title_score_strict(query, title):
    q, t = query.lower(), title.lower()
    return max(
        fuzz.ratio(q, t),
        fuzz.token_sort_ratio(q, t),
    ) / 100.0
```

### **normalize**

La funzione `normalize` esegue una min-max normalization, portando tutti i valori in  $[0, 1]$ . In primis calcola il valore minimo e massimo della serie per poi spostare tutti i valori facendo diventare il minimo 0 ed il massimo 1. Inoltre, se il valore minimo corrisponde al valore massimo restituisce la serie senza alterarla, per evitare di eseguire l'operazione  $\frac{0}{0}$ .

```
def normalize(series):  
    mn, mx = series.min(), series.max()  
    if mx == mn:  
        return pd.Series(np.zeros(len(series)), index=series.index)  
    return (series - mn) / (mx - mn)
```

## Preprocessing del dataset

Per il preprocessing del dataset vengono eseguite diverse operazioni in modo da pulirlo e renderlo più comodo per eseguire la vera e propria ricerca. In primo luogo, viene estratto l'anno di pubblicazione del film dal campo `release_date`, in cui abbiamo una data in formato YYYY-MM-DD, ed inserito nella colonna `year`. In questo modo se abbiamo film omonimi possiamo distinguerli in base all'anno di rilascio, piuttosto che scartarli. Successivamente viene ricavato il nome del regista semplicemente applicando alla colonna `crew` la funzione `get_director` definita precedentemente e riportando il risultato all'interno della nuova colonna `director`. Allo stesso modo estraiamo i nomi del cast con la funzione `extract_list` e li uniamo al nome del regista, dato che non ci è utile averli separati.

Eliminiamo le colonne per cui non è stato rilasciato il film controllando la colonna `status` per la stringa "Released", poiché potrebbero contenere dati mancanti o inesatti.

Vengono poi eliminate tutte quelle colonne che non sono utili alla nostra ricerca, ovvero: `title_y` (duplicata in seguito al merge), `tagline`, `homepage`, `runtime`, `budget`, `revenue`, `vote_count`, `vote_average`, `popularity`, `movie_id` (usiamo la colonna `id` al suo posto), `production_countries`, `spoken_languages`, `production_companies`, `director` (non più utile visto che abbiamo il suo nome insieme al cast) e `status`. Dato che nella colonna `overview` è contenuta la sinossi del film è un dato preziosissimo per la ricerca tramite query, scartiamo le due righe che non presentano questa descrizione. Rinominiamo poi la colonna `title_x` per farla tornare al nome originale `title`.

Come abbiamo fatto per le altre colonne estraiamo anche i contenuti delle colonne `genres` e `keywords` per eliminare il formato JSON-like e poterli usare nella nostra ricerca. Viene poi sostituito il contenuto delle righe vuote nelle colonne che contengono stringhe con stringhe vuote per evitare errori durante la ricerca o l'utilizzo di qualche funzione.

Uniamo poi il contenuto di tutte le colonne che contengono testi e tag che possono aiutarci nella ricerca descrittiva di un film e andiamo a salvare l'unione di questi testi nella nuova colonna `semantic_text`. Tutto questo facendo particolare attenzione a preservare i contenuti così come sono e senza eseguire stemming dato che i sentence transformers sono addestrati su testo leggibile e deteriorano i propri risultati con forme ridotte.

Infine, reimpostiamo gli indici del dataframe per evitare il disallineamento degli indici.

```
df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")
df["year"] = df["release_date"].dt.year.astype("Int64")

df["director"] = df["crew"].apply(get_director)
df["cast"] = df["cast"].apply(lambda x: extract_list(x, limit=CAST_LIMIT))
df["cast"] = df["cast"] + df["director"].apply(lambda x: [x])

df = df[df["status"] == "Released"]

df = df.drop(["title_y", "tagline", "homepage", "runtime", "budget", "revenue",
"vote_count", "vote_average", "popularity", "movie_id", "production_countries",
"spoken_languages", "production_companies", "director", "status"], axis = 1)
df = df.dropna(subset=["overview"])
df = df.rename(columns={"title_x": "title"})

df["genres"] = df["genres"].apply(extract_list)
df["keywords"] = df["keywords"].apply(extract_names)
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].fillna("")

df["semantic_text"] = (
    df["overview"] + " " +
    df["genres"].apply(lambda x: " ".join(x) if isinstance(x, list) else str(x)) + " " +
    df["keywords"] + " " +
    df["cast"].apply(lambda x: " ".join(x) if isinstance(x, list) else str(x))
)

df = df.reset_index(drop=True)
```

## Encoding del database

Arrivati a questo punto utilizziamo il modello all-mpnet-base-v2 scelto precedentemente per eseguire l'encoding del nostro semantic\_text con i sentence transformers. Tramite il parametro batch\_size=64 il modello elabora 64 testi alla volta in modo da essere più veloce.

La stessa operazione viene eseguita anche sui titoli in modo da poterli classificare usando la funzione con la Cosine Similarity piuttosto che sul Fuzzy Search.

```

sentence_model = SentenceTransformer(MODEL_NAME)

print("Encoding del database in corso...")
db_vectors = sentence_model.encode(
    df["semantic_text"].tolist(),
    show_progress_bar=True,
    batch_size=64,
)
print("Encoding completato.\n")

print("Encoding dei titoli in corso...")
title_vectors = sentence_model.encode(
    df["title"].tolist(),
    show_progress_bar=True,
    batch_size=64,
)
print("Encoding completato.")

```

## Funzione di ricerca

Ora entriamo nel pieno del progetto andando a definire la funzione di ricerca `search` che utilizzeremo per il vero e proprio confronto. Questa funzione, data una query (titolo anche approssimato, o stringa descrittiva), restituisce i TOP film più rilevanti ordinati per score totale. Qui è possibile anche selezionare la `search_mode` preferita tra la modalità fuzzy e la modalità cosine che sfruttano diversi algoritmi per la similarità dei contenuti del testo. Lo scoring viene effettuato tramite dei pesi adattivi e l'aggiunta di un punteggio ulteriore in caso di presenza del nome di un attore o di un genere preciso. A questo punto viene eseguito l'encoding anche della query che abbiamo ricevuto in input per poterla confrontare successivamente in base alla modalità scelta.

Qui entriamo nel vivo delle due modalità, se ci troviamo nella modalità fuzzy le operazioni saranno le seguenti:

Viene calcolata la soglia di somiglianza ai titoli tramite `fuzzy_title_score_strict`. Il valore deve essere maggiore o uguale al valore di `TITLE_THRESHOLD` per poter entrare nella modalità di ricerca tramite titolo, in cui il peso dello score di quest'ultimo è definito nella variabile `WEIGHT_TITLE_HIGH` (0,70), altrimenti sarà `WEIGHT_TITLE_LOW` (0.25). In questo modo il peso dello score della somiglianza semantica sarà complementare a quello dei titoli. A questo punto viene eseguito il vero e proprio ranking dei titoli tramite `fuzzy_title_score` per consentire la rilevanza anche dei titoli parziali.

Se invece ci troviamo in modalità cosine le operazioni saranno le seguenti:

Viene calcolata la cosine similarity tra i vettori dei titoli ed il vettore della query, per poi trasformare la lista degli score appena ottenuti in una Series. Successivamente cerca e



memorizza nella variabile `max_strict` il valore più alto presente in tutta la lista dei punteggi, per capire qual è il grado massimo di pertinenza trovato per quella specifica ricerca. Infine, assegna gli score ai titoli e usa i punteggi ottenuti per definire il peso e la modalità di ricerca da applicare.

Se ci troviamo in modalità descrittiva vengono cercati inoltre nomi completi di attori e generi all'interno della query, in modo da poter aggiungere 0,25 (fino ad 1 per ogni categoria) allo score totale, questo perché si è ritenuto più importante il genere ed il nome di un attore in una ricerca piuttosto che una ricerca sommaria.

L'encoding della query eseguito precedentemente potrà finalmente essere confrontato tramite la Cosine Similarity con i vettori ottenuti dall'encoding del testo semantico. Possiamo a questo punto normalizzare gli score ottenuti sia dalla similarità del titolo sia dalla similarità della semantica, per poi andare a sommare tutti i fattori con i loro rispettivi pesi.

Salviamo i nostri risultati in un nuovo dataframe chiamato `results`, che, oltre a possedere gli attributi del database originale `title`, `year` e `genres`, ottiene i nuovi attributi relativi ai diversi score calcolati.

Stampa, infine, la modalità di ricerca che è stata utilizzata con i pesi relativi che sono stati usati ed il valore di corrispondenza massima della query in input con i titoli del database. Tutto questo per consentirci di eseguire una piccola fase di debug nel caso in cui stiamo filtrando il database.

```

def search(query, search_mode="fuzzy", top = TOP):
    found_genres = pd.Series(np.zeros(len(df)), index=df.index)
    found_actors = pd.Series(np.zeros(len(df)), index=df.index)

    q_vec = sentence_model.encode([query])

    if search_mode == "fuzzy":
        strict_scores = df["title"].apply(lambda t: fuzzy_title_score_strict(query, t))
        max_strict = strict_scores.max()
        title_scores = df["title"].apply(lambda t: fuzzy_title_score(query, t))
    else:
        title_cos = cosine_similarity(title_vectors, q_vec).flatten()
        strict_scores = pd.Series(title_cos, index=df.index)
        max_strict = strict_scores.max()
        title_scores = strict_scores

    if max_strict >= TITLE_THRESHOLD:
        w_title = WEIGHT_TITLE_HIGH
        mode = "titolo"
    else:
        w_title = WEIGHT_TITLE_LOW
        mode = "descrittiva"
        df["genre_found"] = df["genres"].apply(lambda genres: [g for g in genres if g.lower() in query.lower()])
        df["genre_score_found"] = df["genre_found"].apply(lambda x: len([i for i in x if i]) * 0.25)
        found_genres = df["genre_score_found"].clip(upper=1.0)

        df["cast_found"] = df["cast"].apply(lambda names: [n for n in names if n.lower() in query.lower()])
        df["cast_score_found"] = df["cast_found"].apply(lambda x: len([i for i in x if i]) * 0.25)
        found_actors = df["cast_score_found"].clip(upper=1.0)

    w_semantic = 1.0 - w_title

    sem_scores = cosine_similarity(db_vectors, q_vec).flatten()

    sem_norm = normalize(pd.Series(sem_scores, index=df.index))
    title_norm = normalize(pd.Series(title_scores, index=df.index))
    total = w_semantic * sem_norm + w_title * title_norm + found_genres

    results = df[["title", "year", "genres"]].copy()
    results["score_semantico"] = sem_norm.values
    results["score_titolo"] = title_norm.values
    results["cast_score_found"] = found_actors.values
    results["genre_score_found"] = found_genres.values
    results["score_totale"] = total.values

    if top != len(df):
        print(
            f"[search] modalità: {mode} | "
            f"w_titolo={w_title:.2f} | w_semantico={w_semantic:.2f} | "
            f"max_strict={max_strict:.2f}"
        )

    return results.sort_values("score_totale", ascending=False).head(top)

```

## Esecuzione

A questo punto non resta che l'esecuzione del codice andando a richiamare la funzione search, prima in modalità fuzzy e poi per la modalità cosine, per le diverse query che diamo in input e stampare i risultati per verificare il corretto funzionamento del codice.

```

for query in queries:
    query = query[0]
    top = search(query, search_mode="fuzzy")

    print(f'\nTop {TOP} film per: "{query}"\n')
    print(f'{"#":<3} {"Titolo":<45} {"Anno":<6} {"Totale":>7} {"Semantico":>9} {"Titolo":>7} {"Genere":>7} {"Cast":>7} {"Generi":<10}')
    print("-" * 135)
    for rank, (_, row) in enumerate(top.iterrows(), start=1):
        print(
            f'{rank:<3} {str(row["title"])[44:<45]} {str(row["year"]):<6} '
            f'f'{row["score_totale"]:>7.3f} {row["score_semantico"]:>9.3f} {row["score_titolo"]:>7.3f} '
            f'{row["genre_score_found"]:>7.3f} {row["cast_score_found"]:>7.3f} {str(", ".join(row["genres"][:3])):<10}'
        )

```

[search] modalità: titolo | w\_titolo=0.70 | w\_semantico=0.30 | max\_strict=1.00

Top 10 film per: "Blade Runner"

| #  | Titolo                         | Anno | Totale | Semantico | Titolo | Genere | Cast  | Generi                           |
|----|--------------------------------|------|--------|-----------|--------|--------|-------|----------------------------------|
| 1  | Blade Runner                   | 1982 | 1.000  | 1.000     | 1.000  | 0.000  | 0.000 | Science Fiction, Drama, Thriller |
| 2  | Blade                          | 1998 | 0.902  | 0.673     | 1.000  | 0.000  | 0.000 | Horror, Action                   |
| 3  | Runner Runner                  | 2013 | 0.895  | 0.650     | 1.000  | 0.000  | 0.000 | Crime, Thriller, Drama           |
| 4  | Blade II                       | 2002 | 0.806  | 0.679     | 0.860  | 0.000  | 0.000 | Fantasy, Horror, Action          |
| 5  | The Maze Runner                | 2014 | 0.749  | 0.582     | 0.820  | 0.000  | 0.000 | Action, Mystery, Science Fiction |
| 6  | The Kite Runner                | 2007 | 0.733  | 0.576     | 0.800  | 0.000  | 0.000 | Drama                            |
| 7  | Nine                           | 2009 | 0.728  | 0.676     | 0.750  | 0.000  | 0.000 | Drama, Music, Romance            |
| 8  | Blade: Trinity                 | 2004 | 0.726  | 0.787     | 0.700  | 0.000  | 0.000 | Science Fiction, Action, Horror  |
| 9  | Maze Runner: The Scorch Trials | 2015 | 0.722  | 0.588     | 0.780  | 0.000  | 0.000 | Action                           |
| 10 | Blue Ruin                      | 2014 | 0.709  | 0.542     | 0.780  | 0.000  | 0.000 | Crime, Thriller                  |

Top 10 film per: "science fiction movie with light sabers"

| #  | Titolo                                       | Anno | Totale | Semantico | Titolo | Genere | Cast  | Generi                             |
|----|----------------------------------------------|------|--------|-----------|--------|--------|-------|------------------------------------|
| 1  | Star Wars                                    | 1977 | 1.170  | 0.989     | 0.711  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 2  | Return of the Jedi                           | 1983 | 1.128  | 0.970     | 0.602  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 3  | Star Wars: Episode I – The Phantom Menace    | 1999 | 1.127  | 1.000     | 0.506  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 4  | Ultramarines: A Warhammer 40,000 Movie       | 2010 | 1.090  | 0.939     | 0.542  | 0.250  | 0.000 | Animation, Science Fiction         |
| 5  | The Empire Strikes Back                      | 1980 | 1.062  | 0.914     | 0.506  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 6  | Star Wars: Clone Wars: Volume 1              | 2005 | 1.040  | 0.889     | 0.494  | 0.250  | 0.000 | Action, Adventure, Animation       |
| 7  | Pacific Rim                                  | 2013 | 1.033  | 0.851     | 0.578  | 0.250  | 0.000 | Action, Science Fiction, Adventure |
| 8  | Star Wars: Episode II – Attack of the Clones | 2002 | 1.024  | 0.868     | 0.494  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 9  | Blade: Trinity                               | 2004 | 1.024  | 0.859     | 0.518  | 0.250  | 0.000 | Science Fiction, Action, Horror    |
| 10 | Star Wars: Episode III – Revenge of the Sith | 2005 | 1.019  | 0.861     | 0.494  | 0.250  | 0.000 | Science Fiction, Adventure, Action |

```

for query in queries:
    query = query[0]
    top = search(query, search_mode="cosine")

    print(f'\nTop {TOP} film per: "{query}"\n')
    print(f'{"#":<3} {"Titolo":<45} {"Anno":<6} {"Totale":>7} {"Semantico":>9} {"Titolo":>7} {"Genere":>7} {"Cast":>7} {"Generi":<10}')
    print("-" * 135)
    for rank, (_, row) in enumerate(top.iterrows(), start=1):
        print(
            f'{rank:<3} {str(row["title"])[44:<45]} {str(row["year"]):<6} '
            f'f'{row["score_totale"]:>7.3f} {row["score_semantico"]:>9.3f} {row["score_titolo"]:>7.3f} '
            f'{row["genre_score_found"]:>7.3f} {row["cast_score_found"]:>7.3f} {str(", ".join(row["genres"][:3])):<10}'
        )

```

[search] modalità: titolo | w\_titolo=0.70 | w\_semantico=0.30 | max\_strict=1.00

Top 10 film per: "Blade Runner"

| #  | Titolo                 | Anno | Totale | Semantico | Titolo | Genere | Cast  | Generi                            |
|----|------------------------|------|--------|-----------|--------|--------|-------|-----------------------------------|
| 1  | Blade Runner           | 1982 | 1.000  | 1.000     | 1.000  | 0.000  | 0.000 | Science Fiction, Drama, Thriller  |
| 2  | Terminator Salvation   | 2009 | 0.703  | 0.726     | 0.693  | 0.000  | 0.000 | Action, Science Fiction, Thriller |
| 3  | Sicario                | 2015 | 0.695  | 0.748     | 0.673  | 0.000  | 0.000 | Action, Crime, Drama              |
| 4  | Blade II               | 2002 | 0.692  | 0.679     | 0.698  | 0.000  | 0.000 | Fantasy, Horror, Action           |
| 5  | Interstellar           | 2014 | 0.676  | 0.595     | 0.710  | 0.000  | 0.000 | Adventure, Drama, Science Fiction |
| 6  | Tomorrowland           | 2015 | 0.669  | 0.712     | 0.650  | 0.000  | 0.000 | Adventure, Family, Mystery        |
| 7  | Blade: Trinity         | 2004 | 0.663  | 0.787     | 0.611  | 0.000  | 0.000 | Science Fiction, Action, Horror   |
| 8  | Shutter Island         | 2010 | 0.654  | 0.621     | 0.669  | 0.000  | 0.000 | Drama, Thriller, Mystery          |
| 9  | The Matrix Revolutions | 2003 | 0.647  | 0.684     | 0.631  | 0.000  | 0.000 | Adventure, Action, Thriller       |
| 10 | Reservoir Dogs         | 1992 | 0.645  | 0.518     | 0.700  | 0.000  | 0.000 | Crime, Thriller                   |

Top 10 film per: "science fiction movie with light sabers"

| #  | Titolo                                       | Anno | Totale | Semantico | Titolo | Genere | Cast  | Generi                             |
|----|----------------------------------------------|------|--------|-----------|--------|--------|-------|------------------------------------|
| 1  | Star Wars: Episode I – The Phantom Menace    | 1999 | 1.229  | 1.000     | 0.915  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 2  | Return of the Jedi                           | 1983 | 1.228  | 0.970     | 1.000  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 3  | Star Wars                                    | 1977 | 1.189  | 0.989     | 0.789  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 4  | The Empire Strikes Back                      | 1980 | 1.157  | 0.914     | 0.884  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 5  | Star Wars: Episode III – Revenge of the Sith | 2005 | 1.125  | 0.861     | 0.917  | 0.250  | 0.000 | Science Fiction, Adventure, Action |
| 6  | Ultramarines: A Warhammer 40,000 Movie       | 2010 | 1.101  | 0.939     | 0.585  | 0.250  | 0.000 | Animation, Science Fiction         |
| 7  | Star Wars: Episode II – Attack of the Clones | 2002 | 1.087  | 0.868     | 0.747  | 0.250  | 0.000 | Adventure, Action, Science Fiction |
| 8  | Star Wars: Clone Wars: Volume 1              | 2005 | 1.066  | 0.889     | 0.598  | 0.250  | 0.000 | Action, Adventure, Animation       |
| 9  | Blade: Trinity                               | 2004 | 1.054  | 0.859     | 0.637  | 0.250  | 0.000 | Science Fiction, Action, Horror    |
| 10 | Pacific Rim                                  | 2013 | 1.031  | 0.851     | 0.571  | 0.250  | 0.000 | Action, Science Fiction, Adventure |

## Validazione

Per validare l'approccio migliore eseguiamo nuovamente per tutte le query il nostro codice, andando a verificare il loro funzionamento tramite la posizione del film che ci aspettiamo di ottenere. Questo viene fatto andando a contare le varie volte in cui l'indice di un algoritmo è posizionato più in alto del suo competitor.

```
fuzzy_counter = 0
cosine_counter = 0
same_counter = 0

print(f'{"Query":<45} {"Titolo ricercato":<45} {"Fuzzy":>7} {"Cosine":>7}')
print("-" * 108)
for query in queries:
    try:
        fuzzy_results = search(query[0], search_mode="fuzzy", top=len(df)).reset_index(drop=True)
        fuzzy_position = fuzzy_results[fuzzy_results["title"].str.lower() == query[1].lower()].index[0] + 1

        cosine_results = search(query[0], search_mode="cosine", top=len(df)).reset_index(drop=True)
        cosine_position = cosine_results[cosine_results["title"].str.lower() == query[1].lower()].index[0] + 1
    except IndexError:
        print(f"Film {query[1]} non trovato.")

    if fuzzy_position < cosine_position:
        fuzzy_counter += 1
    elif fuzzy_position > cosine_position:
        cosine_counter += 1
    else:
        same_counter += 1

    print(f'{query[0][:44]:<45} {query[1][:44]:<45} {fuzzy_position:>7} {cosine_position:>7}')
```

print("\nRisultati:\n")
 f"Fuzzy migliore in {fuzzy\_counter}/25 casi\n"
 f"Cosine migliore in {cosine\_counter}/25 casi\n"
 f"Pari in {same\_counter}/25 casi"
)

| Query                                        | Titolo ricercato                             | Fuzzy | Cosine |
|----------------------------------------------|----------------------------------------------|-------|--------|
| Blade Runner                                 | Blade Runner                                 | 1     | 1      |
| The Dark Knight Rises                        | The Dark Knight Rises                        | 1     | 1      |
| Raging Bull                                  | Raging Bull                                  | 1     | 1      |
| Robocop                                      | Robocop                                      | 1     | 1      |
| goffather                                    | The Godfather                                | 4     | 4046   |
| gost                                         | Ghost                                        | 6     | 2513   |
| lord of the rings                            | The Lord of the Rings: The Fellowship of the | 1     | 1      |
| montecristo                                  | The count of monte cristo                    | 2     | 442    |
| the mtrax                                    | The Matrix                                   | 5     | 222    |
| big short                                    | The Big Short                                | 1     | 1      |
| science fiction movie with light sabers      | Star Wars                                    | 1     | 3      |
| animated movie about a rat who cooks         | Ratatouille                                  | 1     | 1      |
| superhero with iron suit                     | Iron Man                                     | 1     | 1      |
| killer shark on a beach town                 | Jaws                                         | 4     | 6      |
| space crew finds alien eggs on a planet      | Alien                                        | 1     | 1      |
| mel brooks parody of star wars with spaceshi | Spaceballs                                   | 1     | 1      |
| time travel DeLorean 1985                    | Back to the Future                           | 1     | 1      |
| tributes are chosen to compete in killing ga | The Hunger Games                             | 4     | 3      |
| horror movie with a masked killer            | Halloween                                    | 1     | 1      |
| cyborg travels into the past to kill         | The Terminator                               | 1     | 1      |
| emma stone, ryan gosling and steve carell    | Crazy, Stupid, Love.                         | 1     | 1      |
| matt damon and robin williams                | Good Will Hunting                            | 1     | 1      |
| ryan gosling as a stuntman                   | Drive                                        | 1     | 1      |
| margot robbie in a martin scorsese movie     | The Wolf of Wall Street                      | 1     | 1      |
| science fiction movie by john carpenter      | The Thing                                    | 1     | 1      |
| Risultati:                                   |                                              |       |        |
| Fuzzy migliore in 6/25 casi                  |                                              |       |        |
| Cosine migliore in 1/25 casi                 |                                              |       |        |
| Pari in 18/25 casi                           |                                              |       |        |

Come è possibile notare nella maggior parte dei casi si verifica un pareggio fra i due approcci, tuttavia, in 6 casi su 25 è l'algoritmo di fuzzy search a vincere il duello. Questa cosa è particolarmente visibile nella ricerca dei titoli parziali o negli errori di battitura, in cui l'approccio dato dalla cosine arriva in posizioni estremamente basse rispetto alla ricerca fuzzy.

È dunque a mio parere l'algoritmo fuzzy ad avere la meglio sul rivale, e risulta più valido rispetto all'approccio tramite cosine.

### Metriche di valutazione

Per valutare i modelli, oltre la definizione della miglior posizione in classifica, possiamo utilizzare due metriche importanti: Mean Reciprocal Rank (MRR) e Hit@K. Per calcolare queste metriche definiamo la funzione evaluate.

Il Mean Reciprocal Rank è un indice statistico usato per valutare un processo che produce una lista di possibili risposte ad una interrogazione, ordinate per probabilità di correttezza. Si calcola semplicemente facendo:

$$MRR = \frac{\sum_{i=1}^Q \frac{1}{rank_i}}{Q}$$

Dove scorriamo da 1 a Q le query del nostro insieme.

L'Hit@K, invece, controlla se il film atteso è presente nei top-K risultati, in questo caso controlliamo per  $K = [1, 5, 10]$ .

```
def evaluate(queries, mode):
    reciprocal_ranks = []
    k_values=[1, 5, 10]
    hits = {k: [] for k in k_values}

    for query, expected_title in queries:
        results = search(query, search_mode=mode, top=len(df))
        titles = results["title"].str.lower().tolist()

        rank = titles.index(expected_title.lower()) + 1

        reciprocal_ranks.append(1 / rank if rank else 0)
        for k in k_values:
            hits[k].append(1 if rank and rank <= k else 0)

    mrr = sum(reciprocal_ranks) / len(reciprocal_ranks)
    hit_k = {k: sum(hits[k]) / len(hits[k]) for k in k_values}
    return {"MRR": round(mrr, 4), **{f"Hit@{k}": round(v, 4) for k, v in hit_k.items()}}
```

Come possiamo vedere da questi score il punteggio della fuzzy resta ancora in vantaggio, i punteggi in cui il delta tra le due modalità è basso è nell'Hit@1 e nell'MRR. Negli altri risultati abbiamo un distacco più significativo.

```
fuzzy_scores = evaluate(queries, mode="fuzzy")
cosine_scores = evaluate(queries, mode="cosine")

print(f'{"Modalità":<10} {"MRR":>8} {"Hit@1":>8} {"Hit@5":>8} {"Hit@10":>8}')
print("-" * 47)
print(f'{"Fuzzy":<10} {fuzzy_scores["MRR"]:>8} {fuzzy_scores["Hit@1"]:>8} {fuzzy_scores["Hit@5"]:>8} {fuzzy_scores["Hit@10"]:>8}')
print(f'{"Cosine":<10} {cosine_scores["MRR"]:>8} {cosine_scores["Hit@1"]:>8} {cosine_scores["Hit@5"]:>8} {cosine_scores["Hit@10"]:>8}')
```

| Modalità | MRR    | Hit@1 | Hit@5 | Hit@10 |
|----------|--------|-------|-------|--------|
| Fuzzy    | 0.8247 | 0.76  | 0.96  | 1.0    |
| Cosine   | 0.7536 | 0.72  | 0.8   | 0.84   |

## Conclusioni

Il progetto ha portato alla realizzazione di un sistema di ricerca e raccomandazione cinematografica funzionante, capace di gestire con efficacia sia ricerche per titolo, anche approssimato o parziale, sia ricerche descrittive a testo libero, su un dataset di oltre 4800 film.

L'analisi comparativa tra i due approcci implementati ha evidenziato come nessuno dei due risulti superiore in assoluto: nella maggior parte dei casi i due algoritmi producono risultati equivalenti. Tuttavia, il fuzzy matching si dimostra più performante in 6 casi su 25, soprattutto nella gestione di titoli parziali e di errori di battitura, scenari in cui la cosine similarity tende a posizionare il film atteso in posizioni molto basse. Tale vantaggio è confermato anche dalle metriche di valutazione formali: il fuzzy matching ottiene punteggi MRR e Hit@K generalmente superiori rispetto all'approccio cosine, con un distacco più netto nelle metriche Hit@5 e Hit@10.

La scelta del modello all-mpnet-base-v2 si è rivelata adeguata alla ricerca semantica descrittiva, garantendo una buona comprensione del significato delle query a fronte di un costo computazionale maggiore rispetto a modelli più leggeri. L'adozione di pesi adattivi nello scoring finale, unita all'aggiunta di bonus per la presenza esplicita di nomi di attori o generi nella query, ha contribuito a rendere il sistema più preciso e flessibile.

Possibili sviluppi futuri potrebbero includere l'integrazione dei due approcci in un sistema ibrido, che sfrutti contemporaneamente i punti di forza del fuzzy matching per le query brevi e quelle della ricerca semantica per le descrizioni articolate, nonché l'estensione del dataset con informazioni più recenti per superare i limiti evidenziati dal brusco calo della produzione registrata dopo il 2015.